МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ
Ордена Трудового Красного Знамени федеральное государственное
Бюджетное образовательное учреждение высшего образования
МОСКОВСКИЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ

# Лабораторная работа № 2

по дисциплине «Математические основы баз данных»

по теме: «Базовые SQL-запросы и фильтрация данных

Выполнил студент группы:
БВТ2505
Соронина Арина Андреевна
Проверил: Быватов Семён Юрьевич

Москва 2026

## Цель и задачи работы:

**Цель**: наполнить спроектированную базу тестовыми данными и составить набор запросов на выборку и фильтрацию. В крупные таблицы (товары, заказы) внести не менее тысячи строк.

**Задачи:**

1. Наполните все таблицы согласованными данными, используя generate_series или иной генератор для массового заполнения.
2. Напишите запрос, выводящий заданные столбцы товаров без дубликатов (DISTINCT), с псевдонимами столбцов.
3. Напишите запрос с фильтрацией по нескольким условиям, объединённым AND и OR со скобками.
4. Примените в запросах предикаты BETWEEN, IN, LIKE (или ILIKE) и IS NULL — каждый хотя бы один раз.
5. Напишите запрос с сортировкой по двум столбцам в разных направлениях.
6. Реализуйте постраничный вывод товаров с помощью LIMIT и OFFSET.
7. Для каждого запроса приведите его текст, назначение и фрагмент результата. которого воссоздаёт схему с нуля.

## Выполнение

*1. Наполнение базы данных «Кинотеатр данными.*

```
TRUNCATE TABLE payments, tickets, bookings, sessions, seats, movie_cast,
               movies, customers, halls, actors, distributors, genres
    RESTART IDENTITY CASCADE;


SELECT setseed(0.42);


INSERT INTO genres (name) VALUES
       ('Драма'), ('Комедия'), ('Боевик'), ('Триллер'), ('Фантастика'),
       ('Фэнтези'), ('Мелодрама'), ('Ужасы'), ('Приключения'), ('Мультфильм');


INSERT INTO distributors (company_name, country) VALUES
       ('Централ Партнершип',   'Россия'),
    ('Каро Премьер',         'Россия'),
    ('Парадиз',              'Россия'),
    ('Нон-Стоп Продакшн',    'Россия'),
    ('Warner Bros.',         'США'),
    ('Universal Pictures',   'США'),
    ('Paramount Pictures',   'США'),
    ('Walt Disney Studios',  'США'),
    ('Sony Pictures',        'США'),
    ('Pathé',                'Франция'),
    ('Toho',                 'Япония'),
    ('Constantin Film',      'Германия');


INSERT INTO actors (full_name, birth_date)
SELECT (ARRAY['Иванов','Петров','Смирнов','Кузнецов','Попов','Соколов',
              'Лебедев','Козлов','Новиков','Морозов','Волков','Фёдоров',
              'Михайлов','Белов','Орлов'])[p.s]
                      || CASE WHEN p.f THEN 'а' ELSE '' END
                      || ' '
                      || CASE WHEN p.f
                                 THEN
(ARRAY['Мария','Анна','Елена','Ольга','Наталья','Юлия'])[p.n]
```

```
                                    ELSE
(ARRAY['Алексей','Дмитрий','Иван','Сергей','Андрей','Павел'])[p.n]

                            END,

                    CASE WHEN random() < 0.10

                            THEN NULL

                            ELSE DATE '1950-01-01' + floor(random() * 20000)::int

                    END

FROM (SELECT random() < 0.5                AS f,

             1 + floor(random() * 15)::int AS s,

             1 + floor(random() * 6)::int  AS n

      FROM generate_series(1, 300)) AS p;


INSERT INTO halls (name, capacity, hall_type) VALUES

    ('Зал 1', 120, '2D'),

    ('Зал 2', 120, '2D'),

    ('Зал 3',  80, '3D'),

    ('Зал 4',  80, '3D'),

    ('Зал 5', 168, 'IMAX'),

    ('Зал 6',  60, '2D');


INSERT INTO seats (hall_id, row_number, seat_number, seat_type)

SELECT h.hall_id,

       r,

       s,

       CASE WHEN r > l.nrows - 2 THEN 'vip' ELSE 'standard' END

FROM halls AS h

JOIN (VALUES ('Зал 1', 10, 12),

             ('Зал 2', 10, 12),

             ('Зал 3',  8, 10),

             ('Зал 4',  8, 10),

             ('Зал 5', 12, 14),

             ('Зал 6',  6, 10)) AS l(hall_name, nrows, nseats)

       ON l.hall_name = h.name

CROSS JOIN LATERAL generate_series(1, l.nrows)  AS r

CROSS JOIN LATERAL generate_series(1, l.nseats) AS s

ORDER BY h.hall_id, r, s;
```

```
INSERT INTO customers (full_name, phone, email)
SELECT (ARRAY['Иванов','Петров','Смирнов','Кузнецов','Попов','Соколов',
              'Лебедев','Козлов','Новиков','Морозов','Волков','Фёдоров',
              'Михайлов','Белов','Орлов'])[p.s]
         || CASE WHEN p.f THEN 'a' ELSE '' END
       || ' '
       || CASE WHEN p.f
              THEN (ARRAY['Мария','Анна','Елена','Ольга','Наталья','Юлия'])[p.n]
              ELSE
(ARRAY['Алексей','Дмитрий','Иван','Сергей','Андрей','Павел'])[p.n]
             END,
       '+7' || (9000000000 + p.g * 7919)::text,
       CASE WHEN random() < 0.20
             THEN NULL
             ELSE 'user' || p.g || '@example.com'
       END
FROM (SELECT g,
                    random() < 0.5                  AS f,
             1 + floor(random() * 15)::int AS s,
             1 + floor(random() * 6)::int  AS n
      FROM generate_series(1, 1000) AS g) AS p;


INSERT INTO movies (title, genre_id, distributor_id, duration_min, age_rating,
release_date)
SELECT
    (ARRAY['Тайна','Ночь','Город','Побег','Возвращение','Легенда',
           'Путь','Тень','Хозяин','Охота','Битва','Зов'])[1   +   floor(random()   *
12)::int]
    || ' ' ||
    (ARRAY['старого замка','забытого острова','третьей планеты','последнего героя',
              'тёмного леса','потерянных душ','северных гор','большого города',
           'золотого века','чужих берегов','вечной зимы','ночного патруля'])[1  +
floor(random() * 12)::int]
       || CASE WHEN random() > 0.25
                    THEN ' ' || (2 + floor(random() * 3)::int)
                    ELSE ''
         END,
```

```
       1 + floor(random() * 10)::int,
       CASE WHEN random() < 0.08
              THEN NULL
              ELSE 1 + floor(random() * 12)::int
       END,
    80 + floor(random() * 100)::int,
    (ARRAY['0+','6+','12+','16+','18+'])[1 + floor(random() * 5)::int],
    DATE '2015-01-01' + floor(random() * 4200)::int
FROM generate_series(1, 1000);

INSERT INTO movie_cast (movie_id, actor_id, role_name)
SELECT m.movie_id,
       ((m.movie_id * 13 + j * 37) % 300) + 1,
       (ARRAY['Главная роль','Второй план','Антагонист','Эпизодическая роль'])[j + 1]
FROM movies AS m
CROSS JOIN generate_series(0, 3) AS j
ORDER BY m.movie_id, j;

INSERT INTO sessions (movie_id, hall_id, start_time, price)
SELECT 1 + floor(random() * 1000)::int,
       h.hall_id,
       TIMESTAMP '2026-09-01 10:00'
           + (k / 4) * INTERVAL '1 day'
           + (k % 4) * INTERVAL '3 hours',
       200 + 50 * floor(random() * 14)::int
           + CASE h.hall_type WHEN 'IMAX' THEN 200
                              WHEN '3D'   THEN 100
                              ELSE 0 END
FROM halls AS h
CROSS JOIN generate_series(0, 499) AS k
ORDER BY k, h.hall_id;

INSERT INTO bookings (customer_id, booking_date, status)
SELECT 1 + floor(random() * 1000)::int,
       TIMESTAMP '2026-08-01' + random() * INTERVAL '60 days',
       CASE WHEN g % 20 < 15 THEN 'confirmed'
```

```
            WHEN g % 20 < 18 THEN 'pending'
            ELSE 'cancelled'
       END
FROM generate_series(1, 2000) AS g
ORDER BY g;

WITH hall_seats AS (
       SELECT seat_id,
            hall_id,
            seat_type,
            ROW_NUMBER() OVER (PARTITION BY hall_id
                               ORDER BY row_number, seat_number) AS ord
    FROM seats
),
plan AS (
       SELECT booking_id,
            (((booking_id - 1) % 1500) * 1237) % 3000 + 1 AS session_id,
            (booking_id - 1) / 1500                       AS blk,
            1 + booking_id % 4                            AS cnt
    FROM bookings
)

INSERT INTO tickets (booking_id, session_id, seat_id, price)
SELECT p.booking_id,
       p.session_id,
       hs.seat_id,
       s.price + CASE hs.seat_type WHEN 'vip' THEN 100 ELSE 0 END
FROM plan AS p
JOIN sessions AS s ON s.session_id = p.session_id
CROSS JOIN LATERAL generate_series(1, p.cnt) AS n
JOIN hall_seats AS hs
     ON hs.hall_id = s.hall_id
    AND hs.ord = p.blk * 30 + n
ORDER BY p.booking_id, n;

INSERT INTO payments (booking_id, amount, payment_method, paid_at)
```

```
SELECT b.booking_id,
       t.total,
       (ARRAY['card','cash','online'])[1 + floor(random() * 3)::int],
       b.booking_date + random() * INTERVAL '30 minutes'
FROM bookings AS b
JOIN (SELECT booking_id, SUM(price) AS total
      FROM tickets
      GROUP BY booking_id) AS t USING (booking_id)
WHERE b.status = 'confirmed'
ORDER BY b.booking_id;
```

2. *Запрос, выводящий заданные столбцы без дубликатов (DISTINCT), с псевдонимами столбцов (AS).*

```
SELECT DISTINCT title AS movie_title
FROM movies
ORDER BY movie_title
```

| | movie_title character varying (200) |
|---|---|
| 1 | Битва большого города |
| 2 | Битва большого города 2 |
| 3 | Битва большого города 3 |
| 4 | Битва большого города 4 |
| 5 | Битва вечной зимы |
| 6 | Битва вечной зимы 3 |
| 7 | Битва забытого острова |
| 8 | Битва забытого острова 2 |
| 9 | Битва забытого острова 3 |
| 10 | Битва золотого века |

3. *Запрос с фильтрацией по нескольким условиям, объединённым AND и OR со скобками.*

```
SELECT movie_id, title, genre_id, age_rating, duration_min, release_date
FROM movies
WHERE (genre_id = 3 OR genre_id = 4)
  AND (age_rating = '16+' OR age_rating = '18+')
  AND release_date >= DATE '2020-01-01'
  AND duration_min >= 120
ORDER BY release_date DESC, title;
```

| | movie_id [PK] integer | title character varying (200) | genre_id integer | age_rating character varying (5) | duration_min integer | release_date date |
|---|---|---|---|---|---|---|
| 1 | 564 | Охота ночного патруля 3 | 3 | 16+ | 153 | 2026-04-30 |
| 2 | 738 | Побег вечной зимы | 3 | 16+ | 178 | 2026-04-02 |
| 3 | 26 | Тень потерянных душ | 3 | 18+ | 164 | 2026-03-05 |
| 4 | 840 | Легенда потерянных ду… | 4 | 16+ | 145 | 2025-12-11 |
| 5 | 904 | Город большого города | 3 | 18+ | 125 | 2025-09-19 |
| 6 | 783 | Город золотого века 3 | 3 | 16+ | 147 | 2025-01-13 |
| 7 | 629 | Легенда большого город… | 4 | 16+ | 154 | 2025-01-10 |
| 8 | 287 | Город ночного патруля 4 | 3 | 16+ | 171 | 2024-12-30 |
| 9 | 215 | Охота северных гор 2 | 3 | 16+ | 175 | 2024-10-13 |
| 10 | 49 | Зов потерянных душ 4 | 3 | 18+ | 175 | 2024-08-18 |

4. *Применение в запросах предикатов BETWEEN, IN, LIKE (или ILIKE) и IS NULL.*

```
SELECT movie_id, title, duration_min
FROM movies
WHERE duration_min BETWEEN 90 AND 120
ORDER BY duration_min, title;
```

| | movie_id [PK] integer | title character varying (200) | duration_min integer |
|---|---|---|---|
| 1 | 880 | Возвращение потерянных душ 3 | 90 |
| 2 | 473 | Город большого города 2 | 90 |
| 3 | 403 | Зов золотого века 2 | 90 |
| 4 | 768 | Охота большого города 3 | 90 |
| 5 | 522 | Побег северных гор 3 | 90 |

```
SELECT movie_id, title, age_rating
FROM movies
WHERE age_rating IN ('0+', '6+', '12+')
ORDER BY age_rating, title;
```

| | movie_id [PK] integer | title character varying (200) | age_rating character varying (5) |
|---|---|---|---|
| 1 | 100 | Битва большого города 3 | 0+ |
| 2 | 949 | Битва вечной зимы 3 | 0+ |
| 3 | 835 | Битва забытого острова 3 | 0+ |
| 4 | 987 | Битва золотого века 2 | 0+ |
| 5 | 159 | Битва последнего героя 3 | 0+ |

```
SELECT movie_id, title, release_date
FROM movies
WHERE title LIKE 'Ночь%'
ORDER BY release_date DESC;
```

| | movie_id [PK] integer | title character varying (200) | release_date date |
|---|---|---|---|
| 1 | 344 | Ночь большого города 2 | 2026-05-14 |
| 2 | 865 | Ночь последнего геро… | 2026-03-06 |
| 3 | 504 | Ночь последнего геро… | 2026-03-03 |
| 4 | 382 | Ночь забытого острова | 2026-02-04 |
| 5 | 884 | Ночь старого замка | 2026-01-31 |

```
SELECT movie_id, title

FROM movies

WHERE title ILIKE '%ночь%'

ORDER BY title;
```

| | movie_id [PK] integer | title character varying (200) |
|---|---|---|
| 1 | 926 | Ночь большого города |
| 2 | 606 | Ночь большого города |
| 3 | 540 | Ночь большого города 2 |
| 4 | 58 | Ночь большого города 2 |
| 5 | 344 | Ночь большого города 2 |

```
SELECT movie_id, title, distributor_id

FROM movies

WHERE distributor_id IS NULL

ORDER BY movie_id;
```

| | movie_id [PK] integer | title character varying (200) | distributor_id integer |
|---|---|---|---|
| 1 | 17 | Ночь потерянных душ 4 | [null] |
| 2 | 22 | Легенда северных гор 2 | [null] |
| 3 | 32 | Тайна старого замка 2 | [null] |
| 4 | 60 | Город северных гор 2 | [null] |
| 5 | 62 | Побег старого замка 4 | [null] |

5. *Запрос с сортировкой по двум столбцам в разных направлениях.*

```
SELECT movie_id, title, genre_id, duration_min

FROM movies

ORDER BY genre_id ASC, duration_min DESC;
```

| | movie_id [PK] integer | title character varying (200) | genre_id integer | duration_min integer |
|---|---|---|---|---|
| 1 | 932 | Ночь потерянных душ | 1 | 179 |
| 2 | 115 | Побег потерянных душ 4 | 1 | 179 |
| 3 | 854 | Легенда большого города 2 | 1 | 179 |
| 4 | 251 | Тайна ночного патруля | 1 | 176 |
| 5 | 832 | Город чужих берегов 4 | 1 | 176 |
| 6 | 642 | Путь потерянных душ 4 | 1 | 176 |
| 7 | 230 | Путь вечной зимы 3 | 1 | 174 |
| 8 | 449 | Зов чужих берегов 3 | 1 | 172 |

6. *Постраничный вывод товаров с помощью LIMIT и OFFSET.*

```
SELECT movie_id, title, release_date

FROM movies

ORDER BY movie_id

LIMIT 20 OFFSET 0;
```

| | movie_id [PK] integer | title character varying (200) | release_date date |
|---|---|---|---|
| 1 | 1 | Битва потерянных душ 4 | 2015-12-31 |
| 2 | 2 | Хозяин третьей планеты 4 | 2016-08-17 |
| 3 | 3 | Тайна третьей планеты 4 | 2023-11-02 |
| 4 | 4 | Зов ночного патруля 3 | 2015-09-07 |
| 5 | 5 | Зов третьей планеты 2 | 2025-11-03 |

```
SELECT movie_id, title, release_date

FROM movies

ORDER BY movie_id

LIMIT 20 OFFSET 20;
```

| | movie_id [PK] integer | title character varying (200) | release_date date |
|---|---|---|---|
| 1 | 21 | Тайна большого города | 2024-04-23 |
| 2 | 22 | Легенда северных гор 2 | 2023-12-22 |
| 3 | 23 | Тень чужих берегов 3 | 2023-05-05 |
| 4 | 24 | Тень ночного патруля 4 | 2024-08-29 |
| 5 | 25 | Битва последнего героя 4 | 2024-08-09 |

## Вывод

База данных кинотеатра наполнена согласованными тестовыми данными (до 5000 строк в таблице). Составлены и выполнены запросы, охватывающие требования методички: выборка без дубликатов с псевдонимами, составные условия с AND/OR и скобками, предикаты BETWEEN, IN, LIKE/ILIKE и IS NULL, сортировка по двум столбцам в разных направлениях и постраничный вывод. Скобки и порядок вычисления условий, трёхзначная логика NULL и необходимость устойчивой сортировки при пагинации показаны на конкретных примерах.